JavaScript Foundations — The Fast Track

Everything in the *Foundations of Programming & JavaScript* course, compressed for a CS graduate — plus the parts the course skips, and the hardest questions you can be asked.

Covers: Computer Science Fundamentals · JavaScript Basics · Conditional Logic & Loops · Arrays · Functions & Modular Thinking · Recursion — then objects, prototypes, `this`, closures, async/event loop, modules, the DOM, and more.

All code outputs and challenge solutions in this document were executed and verified in Node.js 22.

How to use this document

You already know programming. What you need is (1) the **JavaScript-specific** rules that differ from C/Java/Python, (2) the **traps**, and (3) proof that you have mastered them. So each chapter is: *dense summary → traps → exercises*. Parts 7–9 are the exam: output puzzles, coding challenges, and conceptual interview questions, all with answers.

> **Suggested plan (≈ 1–2 weeks instead of ~13 hours of video + assignments)**
>
> 1. Read Parts 1–5 quickly; run every snippet in Node or a browser console.
> 2. Do the course assignments (Temperature Converter, Number Guessing Game, Grade Calculator) yourself *before* reading the reference solutions in Part 5.
> 3. Spend most of your time on Part 6 (beyond the course) and Parts 7–9 (puzzles, challenges, interview questions). Try each before reading the answer.

Contents

1. Computer Science Fundamentals — how JavaScript runs
2. JavaScript Basics: setup, first execution, variables & constants
3. Numbers, Strings, Booleans & the type system
4. Operators: arithmetic, logical, complex expressions, coercion
5. Taking user input + Assignment: Temperature Converter
6. Conditional logic: if/else, switch, ternary
7. Loops: while, do…while, for, for…of, for…in, break/continue/labels
8. Arrays in depth
9. Functions, parameters, return values, scope
10. Closures and recursion
11. Reference solutions to the three course assignments
12. Beyond the course: objects, prototypes, classes, `this`
13. Beyond the course: async JS, promises, the event loop
14. Beyond the course: modules, errors, Map/Set, iterators, generators, Proxy, DOM, tooling
15. Output-prediction puzzles (hard)
16. Coding challenges (hard) with solutions
17. Conceptual interview questions with answers
18. Cheat sheet

Part 1 — Computer Science Fundamentals (JS view)

The course opens with general CS (what a program is, compilers vs interpreters, memory, algorithms). You know that. Here is what is specific to **JavaScript**:

1.1 What JavaScript is

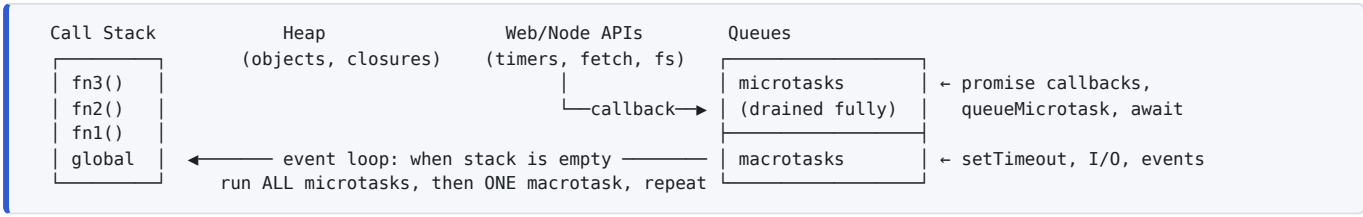
- **Language spec:** ECMAScript (ECMA-262), a new edition every year (ES2015/"ES6" was the big one: `let/const`, arrow functions, classes, modules, promises, destructuring, Map/Set…).
- **Paradigm:** multi-paradigm — imperative, functional (first-class functions, closures), and prototype-based OOP (classes are syntax on top of prototypes).
- **Typing:** dynamically typed, weakly typed (implicit coercion everywhere). TypeScript adds static types on top.
- **Concurrency model:** single-threaded per agent, with an **event loop**; I/O is non-blocking. True parallelism only via Web Workers / worker_threads (no shared variables, message passing; `SharedArrayBuffer` + `Atomics` is the exception).
- **Memory:** garbage-collected (mark-and-sweep, generational in V8). No manual free; leaks come from unintended references (globals, forgotten listeners/timers, closures, caches).

1.2 Engines and runtimes

Concept	Examples	What it provides
Engine	V8 (Chrome, Node, Deno), SpiderMonkey (Firefox), JavaScriptCore (Safari, Bun)	Parsing, compilation, execution of ECMAScript, GC
Runtime / host	Browser, Node.js, Deno, Bun	APIs that are *not* JS: DOM, `fetch`, `setTimeout`, file system, network, the event loop itself

How V8 executes code: source → parser → AST → *Ignition* bytecode interpreter → hot functions are optimized by *TurboFan* (JIT) using type feedback ("hidden classes" / shapes and inline caches). If assumptions break (e.g., a function previously seen only with numbers suddenly gets a string) it **deoptimizes**. Practical consequence: keep object shapes consistent (initialize all properties in the constructor, in the same order) and avoid mixing types in hot code.

1.3 Execution model in one picture



Every function call pushes an **execution context** (its variable environment, `this` binding, and a link to the outer lexical environment). That outer link is what makes **closures** and the **scope chain** work. Deep recursion overflows the stack: `RangeError: Maximum call stack size exceeded`

(≈10k frames in V8, depends on frame size). Proper tail calls exist in the spec but only Safari implements them — do not rely on TCO.

1.4 Compile-time phases you must know

- **Creation phase** (before a scope runs): declarations are registered. `function` declarations are fully initialized (“hoisted” with body). `var` is created and set to `undefined`. `let` / `const` / `class` are created but **uninitialized** — accessing them before the declaration line throws `ReferenceError` (the **Temporal Dead Zone**).
- **Execution phase:** code runs top to bottom.
- **Early errors:** syntax errors abort the whole script before any line runs.

Part 2 — JavaScript Basics

2.0 Setup & first execution (lectures 2.0.1–2.0.2)

- **Browser:** DevTools console (F12). Or an HTML file with `<script src="app.js" defer></script>` (`defer` = run after HTML parsing, in order; `async` = run as soon as downloaded, unordered; `type="module"` is deferred by default).
- **Node.js:** install LTS, run `node app.js`, or `node` for a REPL. `node --watch app.js` re-runs on save.
- **Online:** any playground (the course uses an online compiler). Fine for drills; use VS Code + Node for real work.
- **Output:** `console.log`, `console.error`, `console.table(arrayOfObjects)`, `console.time('x')/timeEnd('x')`, `console.dir(obj, {depth: null})`, `debugger;` statement (breakpoint when DevTools is open).
- Statements end with `;`. **ASI** (automatic semicolon insertion) usually fills them in, but it bites here:

```
function f() {
  return          // ASI inserts ";" here → returns undefined!
  { ok: true };
}
const a = 1
[1, 2].forEach(n => console.log(n))   // parsed as: 1[1,2].forEach → TypeError
const b = 'x'
(function () {})()                    // parsed as: 'x'(function…) → TypeError: "x" is not a
function
```

2.1 Variables and constants

	`var`	`let`	`const`
Scope	function (or global)	block `{ }`	block
Hoisting	hoisted, initialized to `undefined`	hoisted, uninitialized (TDZ)	hoisted, uninitialized (TDZ)
Re-declare in same scope	allowed	SyntaxError	SyntaxError
Re-assign	yes	yes	no (TypeError)
Global creates `window` property	yes (scripts)	no	no
New binding per loop iteration	no	yes	yes (for…of/in)

```
const user = { name: 'Mo' };
user.name = 'Ali';      // OK — const freezes the BINDING, not the value
user = {};              // TypeError: Assignment to constant variable.
Object.freeze(user);    // shallow freeze of the value (nested objects still mutable)

console.log(x); // undefined  (var hoisting)
var x = 5;
console.log(y); // ReferenceError (TDZ)
let y = 5;

z = 10;         // no declaration: creates an implicit global in sloppy mode,
                // ReferenceError in strict mode ('use strict', modules, classes)
```

Rules of thumb

Use `const` by default, `let` when you must reassign, never `var` in new code. Naming: camelCase for variables/functions, PascalCase for classes, UPPER_SNAKE for true constants. Identifiers may contain `$` and `_`, cannot start with a digit, are case-sensitive, and can be Unicode.

Scope types

- **Global**, **module** (each ES module has its own top-level scope), **function**, **block**.
- JavaScript uses **lexical (static) scope**: what a name refers to is decided by where the code is *written*, not where it is called from. (`this` is the exception — it is decided by the call site. See Part 6.)
- **Shadowing:** an inner declaration with the same name hides the outer one.

Part 3 — Numbers, Strings, Booleans & Types

3.1 The type system

8 types: 7 primitives — `undefined`, `null`, `boolean`, `number`, `bigint`, `string`, `symbol` — and `object` (which includes arrays, functions, dates, maps, etc.). Primitives are **immutable** and compared **by value**; objects are compared **by reference**.

Expression	Result	Note
`typeof undefined`	`"undefined"`	
`typeof null`	`"object"`	historic bug, never fixed. Test with `x === null`
`typeof NaN`	`"number"`	NaN is a numeric value
`typeof []`	`"object"`	use `Array.isArray`
`typeof function(){}` / `typeof class{}`	`"function"`	functions are callable objects
`typeof 10n`	`"bigint"`	
`typeof Symbol()`	`"symbol"`	
`typeof undeclaredVar`	`"undefined"`	does not throw (but throws in TDZ!)

Robust type check: `Object.prototype.toString.call(x)` → `"[object Array]"`, `"[object Null]"`, `"[object Date]"`...

`undefined` vs `null`: `undefined` = "no value assigned" (missing variable value, missing property, missing argument, function without return). `null` = "intentionally empty", assigned by the programmer.

3.2 Numbers

- One `number` type: **IEEE-754 double** (64-bit). No separate int. Integers are exact only up to `Number.MAX_SAFE_INTEGER` = $2^{53}-1$ = 9007199254740991.
- `0.1 + 0.2 === 0.30000000000000004`. Compare floats with a tolerance: `Math.abs(a - b) < Number.EPSILON` (scaled for larger magnitudes). For money, use integer cents or BigInt.
- Special values: `NaN` (not equal to anything, including itself), `Infinity`, `-Infinity`, `-0` (`0 === -0` is true; `Object.is(0, -0)` is false; `1/-0 === -Infinity`).
- Division by zero does not throw: `5/0 → Infinity`, `0/0 → NaN`.
- `%` is **remainder**, sign follows the dividend: `-7 % 3 === -1`. True modulo: `((n % m) + m) % m`.
- Literals: `255`, `0xff`, `0o377`, `0b11111111`, `1e6`, `1_000_000` (separators).
- **BigInt:** `2n ** 64n`, arbitrary precision integers. Cannot mix with number (`1n + 1` → TypeError), no `Math.*`, division truncates.

Conversion	Behavior
`Number(s)` / unary `+s`	Whole string must be numeric (whitespace trimmed). `Number('') === 0`, `Number('12px')` → NaN, `Number(null) === 0`, `Number(undefined)` → NaN, `Number([]) === 0`, `Number([5]) === 5`, `Number({})` → NaN
`parseInt(s, radix)`	Parses a prefix: `parseInt('12px') === 12`. **Always pass radix**. `parseInt(0.0000005) === 5` (because String(5e-7) is "5e-7")
`parseFloat(s)`	Prefix float: `parseFloat('12.5px') === 12.5`
`Number.isNaN(x)` vs `isNaN(x)`	`isNaN('abc')` is true (coerces first!). `Number.isNaN('abc')` is false. Prefer `Number.isNaN`, `Number.isFinite`, `Number.isInteger`, `Number.isSafeInteger`
Formatting	`n.toFixed(2)` (returns a string!), `n.toPrecision(3)`, `n.toString(2)`, `new Intl.NumberFormat('en-US',{style:'currency',currency:'USD'}).format(n)`, `n.toLocaleString()`

Math: `Math.round` (rounds .5 toward +∞: `Math.round(-2.5) === -2`), `floor`, `ceil`, `trunc`, `abs`, `max(...arr)`, `min`, `pow`/`**`, `sqrt`, `cbrt`, `hypot`, `sign`, `log2`, `random()` ∈ [0,1).
Random int in [min, max]: `Math.floor(Math.random() * (max - min + 1)) + min`. (Not cryptographically secure — use `crypto.getRandomValues` / `crypto.randomUUID()`.)

3.3 Strings

- Immutable sequences of **UTF-16 code units**. `'😀'.length === 2`! Iterate by code points with `for…of` or `[...str]`; for user-perceived characters use `Intl.Segmenter`.
- Quotes: `'single'`, `"double"`, `` `template ${expr}` `` (multi-line, interpolation). Escapes: `\n \t \\ \' \u{1F600}`.
- `s[0] = 'J'` silently does nothing (throws in strict mode). Every method returns a **new** string.
- Comparison with `<` is lexicographic by code unit: `'10' < '9'` is true, `'B' < 'a'` is true. Use `a.localeCompare(b)` for human sorting.

Need	Method
Access	`s[i]`, `s.at(-1)` (negative index), `s.charAt(i)`, `s.charCodeAt(i)`, `s.codePointAt(i)`
Search	`includes`, `startsWith`, `endsWith`, `indexOf`, `lastIndexOf`, `search(regex)`, `match`, `matchAll`
Extract	`slice(start, end)` (supports negatives), `substring` (swaps args, no negatives), `split(sep, limit)`
Transform	`toUpperCase`, `toLowerCase`, `trim/trimStart/trimEnd`, `padStart(5,'0')`, `padEnd`, `repeat(n)`, `replace` (first match, or all with `/g`), `replaceAll`, `normalize`, `concat`

```
const name = 'Mo', score = 92.456;
console.log(`${name} scored ${score.toFixed(1)}%`);   // "Mo scored 92.5%"
'a-b-c'.split('-');                  // ['a','b','c']
[...'hello'].reverse().join('');     // 'olleh'  (reverse a string)
'Hello'.replace(/l/g, 'L');          // 'HeLLo'
// Tagged template: the function receives the raw parts and values
const tag = (strings, ...vals) => strings.raw.join('|') + ' :: ' + vals;
tag`a${1}b${2}c`;                     // "a|b|c :: 1,2"
```

3.4 Booleans, truthy & falsy

There are exactly **8 falsy values**: `false`, `0`, `-0`, `0n`, `""`, `null`, `undefined`, `NaN`. (Plus the legacy `document.all` in browsers.) **Everything else is truthy**, including `"0"`, `"false"`, `" "`, `[]`, `{}`, `function(){}`, `new Boolean(false)`.

Convert with `Boolean(x)` or `!!x`. Avoid the wrapper objects `new Number/String/Boolean` — they are objects (always truthy).

Autoboxing: `'abc'.length` works because the engine temporarily wraps the primitive in a `String` object to look up the property, then discards it. That's why you can't add properties to primitives: `let s='x'; s.foo=1; s.foo // undefined`.

Part 4 — Operators

4.1 Arithmetic operators (2.3.1)

`+ - * / % **`, unary `+ -`, `++ --` (prefix returns new value, postfix returns old value), compound assignment `+= -= *= /= %= **=`, and the logical assignment operators `&&= ||= ??=`.

The `+` rule: convert both operands to primitives (calls `valueOf` / `toString` / `Symbol.toPrimitive`); if *either* is a string → string concatenation; otherwise numeric addition. All other arithmetic operators always convert to number.

```
1 + '2'        // '12'
'3' - 1        // 2
'3' * '4'      // 12
1 + 2 + '3'    // '33'   (left to right: 3 + '3')
'1' + 2 + 3    // '123'
true + true    // 2
[] + []        // ''
[] + {}        // '[object Object]'
[1,2] + [3]    // '1,23'
'5' - - '2'    // 7
'b' + 'a' + +'a' + 'a'   // 'baNaNa'
2 ** -1        // 0.5;   -2 ** 2 is a SyntaxError (must write (-2) ** 2)
2 ** 3 ** 2    // 512   (** is right-associative)
```

4.2 Comparison & equality

- `===` **strict equality:** no coercion. Only gotchas: `NaN !== NaN` and `+0 === -0`.
- `Object.is`**:** like `===` but `Object.is(NaN, NaN)` true and `Object.is(0, -0)` false. ("SameValue"; `includes`/Map/Set use "SameValueZero": NaN equals NaN, 0 equals -0.)
- `==` **loose equality algorithm** (know it for interviews, avoid it in code — except `x == null` which conveniently means `x === null || x === undefined`):
  1. Same type → compare like `===`.
  2. `null == undefined` → true; `null`/`undefined` equal nothing else (`null == 0` is false).
  3. number vs string → string to number.
  4. boolean vs anything → boolean to number first (`true`→1).
  5. object vs primitive → object to primitive (ToPrimitive), then repeat.
  6. bigint vs number/string → compare mathematically.

```
[] == ![]      // true:  ![] → false → 0 ; [] → '' → 0 ; 0 == 0
[] == false    // true
'' == 0        // true
'0' == false   // true    but  if ('0') { } runs (truthy!)
null == 0      // false   but  null >= 0  is true  (relational ops use ToNumber: null → 0)
undefined == 0 // false;  undefined >= 0 false (NaN)
NaN == NaN     // false
[1] == 1       // true;  [1,2] == '1,2' → true
```

4.3 Logical operators (2.3.2)

`&&`, `||`, `??` **return one of their operands**, not necessarily a boolean, and they **short-circuit**.

Expression	Returns
`a || b`	`a` if `a` is truthy, else `b`
`a && b`	`a` if `a` is falsy, else `b`
`a ?? b`	`a` unless `a` is `null` / `undefined`, then `b`
`!a`	boolean negation
`a?.b`, `a?.[k]`, `f?.()`	`undefined` if `a` is null/undefined (whole chain short-circuits), else the access

```
const port = config.port || 3000;   // BUG if port can be 0
const port2 = config.port ?? 3000;  // correct: only null/undefined fall back
user?.address?.street;              // no TypeError if user or address is missing
obj.count ||= 1;  obj.list ??= [];  // assign only if falsy / nullish
// a || b ?? c   → SyntaxError: must parenthesize when mixing ?? with || or &&
```

4.4 Complex operations (2.3.3): precedence, associativity, other operators

Precedence (high → low, simplified): grouping `()` › member `. [] ?.`, call, `new` › postfix `++ --` › unary `! ~ + - typeof void delete await`, prefix `++ --` › `**` (right-assoc) › `* / %` › `+ -` › shifts `<< >> >>>` › `< <= > >= in instanceof` › `== != === !==` › `&` › `^` › `|` › `&&` › `||` / `??` › `? :` (right-assoc) › assignment (right-assoc) › `,`. **When in doubt, use parentheses.**

- **Bitwise** `& | ^ ~ << >> >>>` convert operands to **32-bit signed ints**. `~~3.7 === 3`, `x | 0` truncates, `(n & 1) === 1` odd test, `n & (n - 1)` clears lowest set bit, `-1 >>> 0 === 4294967295`.
- **Comma** `(a, b)` evaluates both, returns `b`.
- **`void expr`** → always `undefined`.
- **`delete obj.prop`** removes a property (returns boolean); does not delete variables; leaves holes in arrays.
- **`in`**: `'x' in obj` checks own *and inherited* keys; **`instanceof`** checks the prototype chain.
- **Chained comparisons don't work like math:** `1 < 2 < 3` → `true < 3` → `1 < 3` → true; `3 > 2 > 1` → `true > 1` → `1 > 1` → **false**.
- **Increment puzzles:** `let i = 0; i++ + ++i` → `0 + 2 = 2`, and `i` ends at 2.

4.5 Type coercion, formally (ToPrimitive)

When an object is used where a primitive is needed: if it has `[Symbol.toPrimitive](hint)`, call it. Otherwise, for hint `"number"` (or `"default"`) try `valueOf()` then `toString()`; for hint `"string"` try `toString()` first. Plain objects' `valueOf` returns the object itself (not primitive), so `toString` wins → `"[object Object]"`. Arrays' `toString` is `join(',')`. `Date` uses hint "string" for `+`.

```
const money = {
  amount: 42,
  [Symbol.toPrimitive](hint) { return hint === 'number' ? this.amount : `$${this.amount}`; }
};
+money;        // 42
`${money}`;    // '$42'
money + 1;     // '$421'   (hint "default" → our function returned the string)
```

Part 5 — User Input, Control Flow, Loops, Arrays, Functions

5.1 Taking user input (2.4)

```
// Browser — blocking dialogs (fine for exercises, never for real UIs)
const raw = prompt('Enter a temperature:');   // string, or null if cancelled
const n = Number(raw);
if (raw === null || raw.trim() === '' || Number.isNaN(n)) alert('Invalid number');

// Browser — real way: read form inputs
// <input id="t" type="number"> <button id="go">Convert</button>
document.querySelector('#go').addEventListener('click', () => {
  const value = document.querySelector('#t').valueAsNumber;  // NaN if empty
});

// Node.js — command-line arguments
// node app.js 100 C   →  process.argv = ['/path/node', '/path/app.js', '100', 'C']
const [, , value, unit] = process.argv;

// Node.js — interactive input with promises (Node 17+)
const readline = require('node:readline/promises');
const rl = readline.createInterface({ input: process.stdin, output: process.stdout });
const answer = await rl.question('Your name? ');   // top-level await requires an ES module
(.mjs)
rl.close();
// Synchronous alternative many courses use: npm i prompt-sync → const prompt = require('prompt-
sync')();
```

Validation is the real lesson

Input is always a string (or `null`). Always: trim, convert, check `Number.isNaN` / `Number.isFinite`, check ranges, and handle cancel/empty. Assignments are graded largely on this.

5.2 Conditional logic (3.1)

```
if (score >= 90) grade = 'A';
else if (score >= 80) grade = 'B';
else grade = 'F';

const label = age >= 18 ? 'adult' : 'minor';     // ternary is an EXPRESSION

switch (day) {                     // compares with === (strict)!
  case 'sat':
  case 'sun':                      // fall-through groups cases
    type = 'weekend';
    break;                         // forget break → falls into next case
  default:
    type = 'weekday';
}
switch (true) {                    // range matching idiom
  case score >= 90: grade = 'A'; break;
  case score >= 80: grade = 'B'; break;
}
// Object lookup instead of switch
const handlers = { add: (a, b) => a + b, sub: (a, b) => a - b };
const result = handlers[op]?.(a, b) ?? 'unknown op';
```

- Prefer **guard clauses** (early returns) to deep nesting.
- `switch` cases share one block scope: `case 1: let x = 1; break; case 2: let x = 2;` → SyntaxError. Wrap each case in `{ }`.
- `if (x = 5)` is an assignment (always truthy) — a classic bug.

5.3 Loops (3.2, 3.4, 3.5)

```
while (cond) { … }                 // 0+ iterations
do { … } while (cond);             // 1+ iterations — ideal for "ask until valid"
for (let i = 0; i < n; i++) { … }  // classic
for (const value of iterable) { }  // VALUES of arrays, strings, Maps, Sets, generators…
for (const key in object) { }      // enumerable STRING KEYS incl. inherited — not for arrays
arr.forEach((v, i) => { … });      // cannot break; ignores returned values; skips holes

outer: for (const row of grid) {   // labels let break/continue target an outer loop
  for (const cell of row) {
    if (cell === target) break outer;
    if (cell < 0) continue outer;
  }
}
for (const [i, v] of arr.entries()) { }     // index + value
for (const [k, v] of Object.entries(obj)) { }
```

- **`for…in` on arrays** gives string indices, includes inherited/added enumerable props, and order is not guaranteed for non-index keys. Use `for…of`.
- **Closures in loops**: `for (var i…) setTimeout(() => log(i))` logs the final value 3 times; `let` creates a fresh binding per iteration and logs 0,1,2.
- Floating loop counters: `for (let x = 0; x !== 1; x += 0.1)` never terminates.
- Modifying an array while iterating it (splice inside forEach) skips elements — iterate backwards or build a new array.
- Infinite loops block the event loop: the page freezes, timers never fire.

5.4 Arrays (3.3)

Arrays are objects with integer-like keys and a magic `length` (= highest index + 1). They are dynamic, heterogeneous, and can be **sparse** (holes).

Mutating (change the array)	Non-mutating (return new value)
`push/pop` (end, O(1)), `unshift/shift` (start, O(n)), `splice(start, deleteCount, ...items)`, `sort`, `reverse`, `fill`, `copyWithin`, setting `length`	`map`, `filter`, `reduce/reduceRight`, `slice`, `concat`, `flat`, `flatMap`, `join`, `toSorted`, `toReversed`, `toSpliced`, `with(i, v)` (ES2023)

Search	Test	Create
`indexOf` (uses ===, can't find NaN), `includes` (finds NaN), `find`, `findIndex`, `findLast`, `findLastIndex`, `at(-1)`	`some`, `every` (true on empty array!), `Array.isArray`	`[]`, `Array.of(3)` → [3], `Array(3)` → 3 holes, `Array.from({length: 3}, (_, i) => i)`, `Array.from('abc')`, `[...set]`, `Object.groupBy(arr, fn)`

```
// sort() converts to STRINGS by default and is in-place (stable since ES2019)
[10, 1, 5, 100].sort();                 // [1, 10, 100, 5]  ← trap
[10, 1, 5, 100].sort((a, b) => a - b);  // [1, 5, 10, 100]
users.sort((a, b) => a.name.localeCompare(b.name) || b.age - a.age);  // multi-key

['1', '2', '3'].map(parseInt);          // [1, NaN, NaN]  — map passes (value, index):
parseInt('2', 1)
['1', '2', '3'].map(Number);            // [1, 2, 3]

const total = cart.reduce((sum, item) => sum + item.price * item.qty, 0);  // ALWAYS give an
initial value
const byId = Object.fromEntries(users.map(u => [u.id, u]));
const counts = words.reduce((m, w) => (m[w] = (m[w] ?? 0) + 1, m), {});
const unique = [...new Set(arr)];
const chunks = (a, n) => Array.from({ length: Math.ceil(a.length / n) }, (_, i) => a.slice(i * n,
i * n + n));

// Destructuring & spread
const [first, , third = 0, ...rest] = [1, 2, 3, 4, 5];
[a, b] = [b, a];                         // swap
const merged = [...a1, ...a2];

// Copies: spread/slice/Array.from are SHALLOW. Deep copy: structuredClone(value)
const grid = Array(3).fill([]);          // trap: 3 references to the SAME array
const grid2 = Array.from({ length: 3 }, () => []);   // correct

const arr = [1, 2, 3];
arr[10] = 5;   // arr.length === 11, indices 3..9 are holes
arr.length = 0;  // truncates (empties) the array in place
```

Complexity: index access O(1); push/pop O(1) amortized; shift/unshift/splice O(n); `includes/indexOf` O(n) — use a `Set` for repeated membership tests. `sort` is O(n log n) (TimSort in V8).

5.5 Functions (4.1–4.3)

```
function add(a, b) { return a + b; }          // declaration: hoisted with body
const sub = function (a, b) { return a - b; }; // expression: not callable before this line
const mul = (a, b) => a * b;                   // arrow: implicit return
const mk = () => ({ ok: true });               // return an object literal: wrap in ( )
const fact = function f(n) { return n <= 1 ? 1 : n * f(n - 1); }; // named function expression

function greet(name = 'guest', greeting = `Hi ${name}`) { }  // defaults (evaluated at call time,
can use earlier params)
function sum(...nums) { return nums.reduce((s, n) => s + n, 0); } // rest params (real array)
sum(...[1, 2, 3]);                             // spread arguments
function draw({ x = 0, y = 0, color = 'black' } = {}) { }        // named/options parameters
```

	function declaration/expression	**arrow function**
`this`	dynamic (depends on call)	lexical (inherited from enclosing scope)
`arguments`	yes (array-like)	no (uses outer one)
usable with `new`	yes	no (TypeError) — no `prototype`
good for	methods, constructors	callbacks, preserving outer `this`

- Arguments are **passed by value**, but for objects the value is a **reference** (“call by sharing”): mutating a parameter's properties is visible to the caller; reassigning the parameter is not.
- Missing arguments are `undefined`; extra ones are ignored. Default parameters apply only for `undefined` (not `null`).
- `fn.length` = number of params before the first default/rest.
- Functions are first-class objects: store them, pass them, return them, attach properties (`fn.cache = …`).
- **Pure functions** (same input → same output, no side effects) are easier to test and memoize. Write small functions that do one thing — that's the “modular thinking” of chapter 4.
- No function overloading: a second declaration with the same name replaces the first.

5.6 Scope and closures (4.3)

A **closure** is a function together with the lexical environment it was created in. The inner function keeps the outer variables **alive** (by reference, not a copy) even after the outer function has returned.

```
function makeCounter() {
  let count = 0;                      // private state
  return {
    inc: () => ++count,
    get: () => count,
  };
}
const c = makeCounter();
c.inc(); c.inc(); c.get();           // 2 — count is inaccessible from outside

// Module pattern (pre-ES-modules) via IIFE
const Bank = (() => {
  let balance = 0;
  return { deposit(n) { balance += n; return balance; } };
})();

// Function factory / partial application
const multiplier = factor => n => n * factor;
const double = multiplier(2);       // double(5) === 10

// once()
function once(fn) {
  let done = false, result;
  return function (...args) {
    if (!done) { done = true; result = fn.apply(this, args); }
    return result;
  };
}
```

Uses: data privacy, factories, memoization, debounce/throttle, event handlers, iterators, partial application. Cost: captured variables can't be garbage-collected while the closure is reachable (possible memory leak).

5.7 Recursion (4.4)

Base case + recursive case that moves toward it. Each call adds a stack frame; JS has no reliable tail-call optimization, so very deep recursion (>~10k) needs an explicit stack or a loop.

```
const factorial = n => (n <= 1 ? 1 : n * factorial(n - 1));

// Naive fib is O(2^n); memoized is O(n)
const fib = (n, memo = new Map()) => {
  if (n < 2) return n;
  if (!memo.has(n)) memo.set(n, fib(n - 1, memo) + fib(n - 2, memo));
  return memo.get(n);
};

// Recursion on trees / nested data
const sumNested = arr => arr.reduce((s, x) => s + (Array.isArray(x) ? sumNested(x) : x), 0);
const depth = node => node ? 1 + Math.max(depth(node.left), depth(node.right)) : 0;

// Backtracking: all subsets
function subsets(nums) {
  const out = [];
  (function go(i, cur) {
    if (i === nums.length) return out.push([...cur]);
    go(i + 1, cur);                     // exclude nums[i]
    cur.push(nums[i]); go(i + 1, cur); cur.pop();   // include nums[i]
  })(0, []);
  return out;
}

// Converting deep recursion to iteration with an explicit stack (DFS)
function dfs(root, visit) {
  const stack = [root];
  while (stack.length) {
    const node = stack.pop();
    visit(node);
    for (const child of [...node.children].reverse()) stack.push(child);
  }
}

// Trampoline: avoid stack overflow for tail-recursive style code
const trampoline = fn => (...args) => { let r = fn(...args); while (typeof r === 'function') r =
r(); return r; };
const sumTo = trampoline(function s(n, acc = 0) { return n === 0 ? acc : () => s(n - 1, acc + n);
});
sumTo(1e6);   // 500000500000 — no stack overflow
```

Part 5b — Reference Solutions for the Course Assignments

Try them first. A strong submission is not just “works on the happy path” — it validates input, separates pure logic from I/O, and is easy to test.

Assignment: Temperature Converter (Module 2)

```
// Pure logic — no I/O, easy to test
function convertTemperature(value, from, to) {
  const toC   = { C: v => v, F: v => (v - 32) * 5 / 9, K: v => v - 273.15 };
  const fromC = { C: v => v, F: v => v * 9 / 5 + 32,   K: v => v + 273.15 };
  if (!(from in toC) || !(to in fromC)) throw new RangeError(`Unknown unit: ${from} or ${to}`);
  if (!Number.isFinite(value)) throw new TypeError('Temperature must be a finite number');
  const celsius = toC[from](value);
  if (celsius < -273.15) throw new RangeError('Below absolute zero');
  return Math.round(fromC[to](celsius) * 100) / 100;
}

// I/O layer (browser version)
function runConverter() {
  const raw = prompt('Temperature (e.g. "100 C to F"):');
  if (raw === null) return;                                  // cancelled
  const match = raw.trim().match(/^(-?\d+(?:\.\d+)?)\s*([CFK])\s*(?:to\s*)?([CFK])$/i);
  if (!match) return alert('Format: <number> <C|F|K> to <C|F|K>');
  const [, num, from, to] = match;
  try {
    alert(`${num}°${from.toUpperCase()} = ${convertTemperature(Number(num), from.toUpperCase(),
to.toUpperCase())}°${to.toUpperCase()}`);
  } catch (err) {
    alert(err.message);
  }
}
// convertTemperature(100, 'C', 'F') → 212 ; convertTemperature(32, 'F', 'K') → 273.15
```

Design points: table-driven conversion through a canonical unit (C) scales to N units with 2N functions instead of N² branches; rounding is done once at the end; errors are thrown from logic and caught in the UI.

Assignment: Number Guessing Game (Module 3)

```
// Node.js version (save as game.mjs, run: node game.mjs)
import readline from 'node:readline/promises';
import { stdin as input, stdout as output } from 'node:process';

const randomInt = (min, max) => Math.floor(Math.random() * (max - min + 1)) + min;

async function play({ min = 1, max = 100, maxAttempts = 7 } = {}) {
  const rl = readline.createInterface({ input, output });
  const secret = randomInt(min, max);
  const guesses = new Set();
  try {
    for (let attempt = 1; attempt <= maxAttempts; attempt++) {
      const raw = await rl.question(`Attempt ${attempt}/${maxAttempts} — guess (${min}-${max}):
`);
      const guess = Number(raw.trim());
      if (!Number.isInteger(guess) || guess < min || guess > max) {
        console.log('Please enter a whole number in range.');
        attempt--;                       // invalid input does not cost an attempt
        continue;
      }
      if (guesses.has(guess)) { console.log('You already tried that.'); attempt--; continue; }
      guesses.add(guess);
      if (guess === secret) { console.log(`Correct! ${attempt} attempt(s).`); return true; }
      console.log(guess < secret ? 'Too low' : 'Too high');
    }
    console.log(`Out of attempts. The number was ${secret}.`);
    return false;
  } finally {
    rl.close();
  }
}
await play();
```

CS-grad bonus: with binary search the optimal player needs at most $\lceil \log_2(100) \rceil = 7$ guesses, which is why 7 attempts is the “fair” limit. Extension: let the computer guess *your* number with binary search and detect when you lie (low > high).

Assignment: Grade Calculator (Module 4)

```
function letterGrade(score) {
  if (typeof score !== 'number' || Number.isNaN(score) || score < 0 || score > 100)
    throw new RangeError(`Invalid score: ${score}`);
  const scale = [[90, 'A'], [80, 'B'], [70, 'C'], [60, 'D']];
  for (const [min, letter] of scale) if (score >= min) return letter;
  return 'F';
}

const average = nums => nums.reduce((sum, n) => sum + n, 0) / nums.length;

function gradeReport(students) {
  const rows = students.map(({ name, scores }) => {
    const avg = average(scores);
    return { name, average: Number(avg.toFixed(2)), grade: letterGrade(avg) };
  });
  const classAverage = Number(average(rows.map(r => r.average)).toFixed(2));
  const top = rows.reduce((best, r) => (r.average > best.average ? r : best));
  const distribution = Object.groupBy(rows, r => r.grade);   // ES2024
  return { rows, classAverage, top: top.name, distribution };
}

console.table(gradeReport([
  { name: 'Ali',  scores: [90, 95, 100] },
  { name: 'Sara', scores: [70, 65, 80] },
]).rows);
// Ali 95 A ; Sara 71.67 C ; classAverage 83.34 ; top 'Ali'
```

Extensions graders love: weighted categories (`{ homework: 0.3, exams: 0.7 }`), dropping the lowest score, +/− grades, handling empty score lists (division by zero → NaN), and unit tests.

Part 6 — Beyond the Course (1): Objects, Prototypes, Classes, `this`

The course stops at functions. Real JavaScript work and interviews start here.

6.1 Objects

```
const key = 'dynamic';
const user = {
  name: 'Mo',
  'full name': 'Mo D',          // quoted key
  [key + 'Key']: 1,              // computed key
  greet() { return `Hi ${this.name}`; },   // method shorthand
  get upper() { return this.name.toUpperCase(); },   // getter
  set upper(v) { this.name = v.toLowerCase(); },     // setter
};
const { name, age = 18, address: { city } = {}, ...others } = user;   // destructuring
const copy = { ...user, name: 'Ali' };   // shallow copy + override (getters are evaluated!)
Object.keys(o); Object.values(o); Object.entries(o); Object.fromEntries(entries);
Object.assign(target, src1, src2);       // shallow, invokes setters on target
Object.hasOwn(o, 'k');                   // own property check (better than o.hasOwnProperty)
structuredClone(o);                      // deep clone (no functions, no prototypes/classes,
handles cycles, Dates, Maps)
JSON.parse(JSON.stringify(o));           // old deep clone: loses undefined, functions, Dates →
strings, NaN → null, fails on cycles
```

Key order: integer-like keys in ascending order first, then string keys in insertion order, then symbols. Keys are always strings or symbols — `obj[1]` and `obj['1']` are the same; an object used as a key becomes `"[object Object]"` (use a `Map`).

Property descriptors: each property has `value, writable, enumerable, configurable` (or `get/set`). `Object.defineProperty(o, 'id', { value: 1, writable: false, enumerable: false })`. Levels of immutability: `Object.preventExtensions` (no new props) ⊂ `Object.seal` (+ no delete) ⊂ `Object.freeze` (+ no writes). All are **shallow**.

6.2 Prototypes

Every object has a hidden `[[Prototype]]` link (`Object.getPrototypeOf(o)`, legacy `o.__proto__`). Property **reads** walk up the chain until found or `null`. Property **writes** create/modify an own property (shadowing), unless a setter or non-writable prop exists up the chain.

```
const animal = { eats: true, walk() { return `${this.name} walks`; } };
const dog = Object.create(animal);   // dog.[[Prototype]] = animal
dog.name = 'Rex';
dog.walk();          // 'Rex walks' — method found on prototype, this = dog
'eats' in dog;       // true;   Object.hasOwn(dog, 'eats') → false

function Person(name) { this.name = name; }    // constructor function (pre-ES6)
Person.prototype.hi = function () { return this.name; };
const p = new Person('Mo');
// What `new` does:
// 1. create obj with [[Prototype]] = Person.prototype
// 2. call Person with this = obj
// 3. if Person returns an object, use that; else return obj
p.__proto__ === Person.prototype;               // true
Person.prototype.constructor === Person;        // true
Object.getPrototypeOf(Person.prototype) === Object.prototype;   // true
Object.getPrototypeOf(Object.prototype) === null;               // end of chain
p instanceof Person;   // checks whether Person.prototype is in p's chain
```

6.3 Classes

```
class Shape {
  static count = 0;                 // static field
  #id;                              // truly private field (hard privacy, not a convention)
  name = 'shape';                   // public instance field (own property on each instance)
  constructor(id) { this.#id = id; Shape.count++; }
  get id() { return this.#id; }
  area() { throw new Error('not implemented'); }   // on prototype
  toString() { return `${this.name}#${this.#id} area=${this.area()}`; }
  static #secret() { return 42; }   // private static method
  static isShape(x) { return #id in x; }   // ergonomic brand check
}
class Circle extends Shape {
  constructor(id, r) {
    super(id);                      // must call super() before using this
    this.r = r;
    this.name = 'circle';
  }
  area() { return Math.PI * this.r ** 2; }      // override
  toString() { return 'O ' + super.toString(); } // call parent method
}
```

- Classes are functions; methods live on `Class.prototype` and are **non-enumerable**; class bodies are always in **strict mode**; class declarations are in the TDZ until evaluated (not hoisted like functions); calling a class without `new` throws.
- Arrow functions as class fields (`handle = () => {…}`) bind `this` per instance — handy for callbacks, but each instance gets its own copy (more memory, not on the prototype, not overridable via `super`).
- Composition over inheritance: prefer small objects/functions combined together; JS supports mixins: `Object.assign(Target.prototype, mixin)` .

6.4 `this` — the four binding rules (+ arrows)

Call form	`this` is	Priority
`new Foo()`	the newly created object	1 (highest)
`fn.call(obj)`, `fn.apply(obj)`, `fn.bind(obj)()`	`obj` (explicit)	2
`obj.fn()`	`obj` (implicit — the object before the dot *at call time*)	3
`fn()`	`undefined` in strict mode; `globalThis` in sloppy mode	4 (default)
arrow function	ignores all rules above: inherits `this` from the enclosing scope at *creation* time	—

```
const obj = {
  name: 'obj',
  regular() { return this?.name; },
  arrow: () => this,                       // this of the MODULE/global scope, not obj
  delayed() { setTimeout(function () { console.log(this?.name); }, 0); },    // lost
  delayedOk() { setTimeout(() => console.log(this.name), 0); },            // 'obj'
};
const f = obj.regular;
f();                        // undefined (strict) — "lost this": the method was detached
[1].map(obj.regular);       // same problem when passing methods as callbacks
const bound = obj.regular.bind(obj);    // permanent; bind again doesn't change it
bound.call({ name: 'x' }); // 'obj' — call cannot override bind (but new can)
fn.call(ctx, a, b);  fn.apply(ctx, [a, b]);   // call = comma args, apply = array
```

Part 6 — Beyond the Course (2): Asynchronous JavaScript

6.5 Callbacks → Promises → async/await

```
// Callback style (Node error-first convention) — leads to "callback hell" / inversion of control
fs.readFile('a.txt', 'utf8', (err, data) => { if (err) return handle(err); /* … */ });

// Promise: a placeholder for a future value. States: pending → fulfilled | rejected (settled
once, immutable)
const wait = ms => new Promise(resolve => setTimeout(resolve, ms));
fetch('/api/user')
  .then(res => { if (!res.ok) throw new Error(res.status); return res.json(); })  // returning a
promise chains it
  .then(user => console.log(user))
  .catch(err => console.error(err))       // catches any error above
  .finally(() => hideSpinner());           // runs either way, passes the value through

// async/await: syntax over promises. An async function ALWAYS returns a promise.
async function loadUser(id) {
  try {
    const res = await fetch(`/api/users/${id}`);
    if (!res.ok) throw new Error(`HTTP ${res.status}`);
    return await res.json();               // "return await" inside try so the catch below sees
rejections
  } catch (err) {
    console.error('failed', err);
    throw err;                             // re-throw or return a fallback
  }
}
```

Combinator	Resolves when	Rejects when
`Promise.all`	all fulfill (array of values, in input order)	first rejection (others keep running!)
`Promise.allSettled`	all settle (`{status, value|reason}`)	never
`Promise.race`	first to settle, either way	first to settle is a rejection
`Promise.any`	first fulfillment	all reject (`AggregateError`)
`Promise.withResolvers()`	returns `{ promise, resolve, reject }` (ES2024)	

```
// Sequential vs parallel — a classic performance bug
for (const id of ids) results.push(await getUser(id));        // N round trips, one after another
const results = await Promise.all(ids.map(id => getUser(id))); // all in parallel

// forEach does NOT await
ids.forEach(async id => { await save(id); });   // fires all; the code after continues
immediately
for (const id of ids) await save(id);            // truly sequential

// Cancellation
const ctrl = new AbortController();
fetch(url, { signal: ctrl.signal });
setTimeout(() => ctrl.abort(), 5000);   // or: fetch(url, { signal: AbortSignal.timeout(5000) })
```

6.6 The event loop, precisely

1. Run the current script (a macrotask) until the call stack is empty.
2. **Drain the microtask queue completely** (promise reactions, `await` continuations, `queueMicrotask`, MutationObserver). Microtasks queued during this step also run now — so a microtask loop can starve rendering.
3. Browser: maybe render (requestAnimationFrame callbacks, style, layout, paint).
4. Take **one** macrotask (timer, I/O callback, UI event, `MessageChannel`) and go to step 2.

Node specifics: `process.nextTick` queue runs before promise microtasks; the loop has phases (timers → pending callbacks → poll (I/O) → check (`setImmediate`) → close). Inside an I/O callback, `setImmediate` always fires before `setTimeout(…, 0)`; in the main module their order is not deterministic.

`setTimeout(fn, 0)` means "at least 0 ms, after the current task and all microtasks" — browsers clamp nested timers to ≥4 ms. `await x` splits the function: everything after it becomes a microtask continuation, even if `x` is not a promise.

```
console.log('1');
setTimeout(() => console.log('2'), 0);
Promise.resolve().then(() => console.log('3')).then(() => console.log('4'));
queueMicrotask(() => console.log('5'));
(async () => { console.log('6'); await null; console.log('7'); })();
process.nextTick(() => console.log('8'));      // Node only
console.log('9');
// Node output: 1 6 9 8 3 5 7 4 2
```

Unhandled rejections
A rejected promise with no handler triggers `unhandledrejection` (browser) and **crashes the process** in Node ≥15. Always end chains with `.catch` or wrap awaits in try/catch.

Part 6 — Beyond the Course (3): The Rest of the Language

6.7 Errors

```
try { risky(); }
catch (err) {                       // `catch {` without binding is allowed
  if (err instanceof TypeError) { /* … */ } else throw err;   // don't swallow unknown errors
}
finally { cleanup(); }              // a `return` in finally OVERRIDES the try's return/throw

class ValidationError extends Error {
  constructor(field, message, options) {
    super(message, options);        // options.cause → error chaining: new Error('x', { cause:
err })
    this.name = 'ValidationError';
    this.field = field;
  }
}
```

Built-ins: `Error`, `TypeError` (wrong type / calling non-function / reading prop of undefined), `ReferenceError` (undeclared / TDZ), `SyntaxError`, `RangeError` (bad length, stack overflow), `URIError`, `EvalError`, `AggregateError`. You can `throw` anything, but always throw `Error` objects (they carry `stack`).

6.8 Map, Set, WeakMap, WeakSet, WeakRef

- **Map**: any key type (objects, NaN), preserves insertion order, `size`, iterable, O(1) average ops. Use it for dictionaries with dynamic keys instead of plain objects (no prototype key collisions like `"__proto__"` or `"constructor"`).
- **Set**: unique values (SameValueZero). ES2025 adds `union`, `intersection`, `difference`, `symmetricDifference`, `isSubsetOf`…
- **WeakMap/WeakSet**: keys must be objects, held **weakly** (don't prevent GC), not iterable, no size. Perfect for caching metadata about objects you don't own, or private data.
- **WeakRef** / **FinalizationRegistry**: advanced, non-deterministic; avoid unless you really need them.

6.9 Symbols, iterators, generators

```
const id = Symbol('id');                // unique, non-string key; hidden from for…in / JSON /
Object.keys
Symbol.for('app.key') === Symbol.for('app.key');  // global registry → true
// Well-known symbols customize language behavior:
// Symbol.iterator, Symbol.asyncIterator, Symbol.toPrimitive, Symbol.hasInstance,
Symbol.toStringTag

// Iterator protocol: an object with next() → { value, done }
class Range {
  constructor(from, to) { Object.assign(this, { from, to }); }
  *[Symbol.iterator]() { for (let i = this.from; i <= this.to; i++) yield i; }   // generator
method
}
[...new Range(1, 4)];                   // [1, 2, 3, 4]

// Generators: pausable functions; lazy, can be infinite
function* naturals() { let n = 0; while (true) yield n++; }
function* take(it, k) { for (const x of it) { if (k-- <= 0) return; yield x; } }
[...take(naturals(), 3)];               // [0, 1, 2]
const g = (function* () { const x = yield 1; console.log('got', x); })();
g.next();      // { value: 1, done: false }
g.next(42);    // logs 'got 42' → { value: undefined, done: true }

// Async iteration
for await (const chunk of stream) { /* … */ }
async function* poll(url) { while (true) { yield await (await fetch(url)).json(); await
wait(1000); } }
```

6.10 Modules

	ES Modules (standard)	**CommonJS (Node legacy)**
Syntax	`import x, { y as z } from './m.js'` / `export default`, `export const`	`const x = require('./m')` / `module.exports = …`
Loading	static, analyzed before execution (tree-shaking), async; `import()` for dynamic	dynamic, synchronous, at runtime
Bindings	**live** read-only bindings	copy of `module.exports` value at require time
Mode	always strict, top-level `this` is `undefined`, top-level `await` allowed	sloppy by default, `this === module.exports`
Enable in Node	`.mjs` or `"type": "module"` in package.json	`.cjs` or default

Modules are singletons: evaluated once, cached, shared by all importers. Circular imports are allowed but may see uninitialized bindings (TDZ errors).

6.11 Strict mode, hoisting summary, misc semantics

- `'use strict'` (automatic in modules and classes): no implicit globals, `this` is `undefined` in plain calls, assignment to read-only props throws, duplicate param names forbidden, `with` forbidden, `arguments` doesn't alias params, octal literals `010` forbidden.

- **`eval`** and `new Function`: avoid (security, performance, scope surprises).
- **Labeled blocks, `with`, `arguments.callee`**: legacy, avoid.
- **Getter/setter vs data properties**, **optional catch binding**, **numeric separators**, **`globalThis`** (universal global object).
- **Dates:** months are 0-based (`new Date(2026, 0, 31)` = Jan 31); `new Date('2026-10-20')` is UTC, `new Date('2026-10-20T00:00')` is local time. `Temporal` is the modern replacement API (shipping in browsers in 2025–2026).
- **JSON:** `JSON.stringify(value, replacer, indent)` drops `undefined`/functions/symbols in objects (→ `null` in arrays), `NaN`/`Infinity` → `null`, Dates → ISO strings, calls `toJSON()`, throws on cycles and BigInt. `JSON.parse(text, reviver)`.
- **Regular expressions:** `/^(\d{3})-(?<rest>\d+)$/u`; flags `g i m s u y d v`; `str.match`, `matchAll`, `regex.test`, `replace` with `$1`/`$<name>`. Global regexes are **stateful** (`lastIndex`) — calling `.test` twice on the same `/g` regex can alternate true/false.

6.12 Proxy & Reflect (metaprogramming)

```
const validated = new Proxy({}, {
  set(target, key, value, receiver) {
    if (key === 'age' && !Number.isInteger(value)) throw new TypeError('age must be an integer');
    return Reflect.set(target, key, value, receiver);    // Reflect mirrors each trap with
default behavior
  },
  get(target, key, receiver) {
    return key in target ? Reflect.get(target, key, receiver) : `no ${String(key)}`;
  },
});
// Traps: get, set, has, deleteProperty, ownKeys, apply, construct, defineProperty,
getPrototypeOf…
// Vue 3 reactivity, MobX, Immer and many ORMs are built on Proxy.
```

6.13 The browser: DOM & events (what your next courses will use)

```
const list = document.querySelector('#todos');           // first match; querySelectorAll → static NodeList
const li = document.createElement('li');
li.textContent = userInput;          // SAFE. innerHTML = userInput → XSS vulnerability
li.classList.add('item');
li.dataset.id = '42';                // data-id="42"
list.append(li);

// Event delegation: one listener on the parent handles all children (even future ones)
list.addEventListener('click', e => {
  const item = e.target.closest('li.item');
  if (!item) return;
  item.classList.toggle('done');
});
// Event flow: capture (window → target) → target → bubble (target → window)
// e.preventDefault() stops default action (form submit, link navigation)
// e.stopPropagation() stops further propagation
// addEventListener(type, fn, { once: true, passive: true, capture: true, signal })

form.addEventListener('submit', async e => {
  e.preventDefault();
  const data = Object.fromEntries(new FormData(form));
  await fetch('/api', { method: 'POST', headers: { 'Content-Type': 'application/json' }, body: JSON.stringify(data) });
});
// Storage: localStorage (strings only, ~5MB, sync), sessionStorage, IndexedDB (async, large), cookies
// Performance: batch DOM reads then writes (avoid layout thrashing); requestAnimationFrame for animation
```

6.14 Memory leaks & performance

- Leaks: accidental globals, forgotten `setInterval`, listeners on removed elements, unbounded caches/Maps, closures capturing large objects, detached DOM nodes. Find them with DevTools → Memory → heap snapshots.
- **Debounce** (run after the user stops: search box) vs **throttle** (run at most once per interval: scroll/resize).
- Heavy CPU work blocks the UI: split it into chunks (`setTimeout` / `scheduler.yield()`) or move it to a Web Worker.

6.15 The ecosystem you're expected to know as an SE

Area	Tools
Runtimes / package managers	Node.js, Deno, Bun · npm, pnpm, yarn · `package.json`, semver (`^1.2.3` = <2.0.0), lockfiles, `npx`
Types	**TypeScript** (learn it right after this course — it's the industry default)
Quality	ESLint, Prettier, `// @ts-check` + JSDoc
Testing	`node:test`, Vitest, Jest, Playwright (E2E)
Build	Vite, esbuild, Rollup, webpack · transpiling with Babel/SWC
Frameworks	React/Next.js, Vue/Nuxt, Svelte, Angular · backend: Express, Fastify, NestJS, Hono

Part 7 — Output-Prediction Puzzles (the hardest kind of JS question)

Cover the answer, predict the output, then check. Unless noted, code runs as a classic (non-module, sloppy) script in Node. Every answer below was executed.

7.1 Types, coercion, numbers

Q1. `console.log(0.1 + 0.2 === 0.3, 0.1 + 0.2);`

`false 0.30000000000000004` — IEEE-754 doubles can't represent 0.1/0.2 exactly.

Q2. `typeof null, typeof NaN, typeof [], typeof class {}, typeof undeclaredVar`

`'object' 'number' 'object' 'function' 'undefined'`

Q3. `[] + [], [] + {}, 1 + '2' - 1, '3' * '4', true + true, [1,2] + [3]`

`'' '[object Object]' 11 12 2 '1,23'` — `1+'2'` = '12', then `'12'-1` = 11.

Q4. `null == undefined, null === undefined, null == 0, null >= 0, NaN == NaN, Object.is(NaN, NaN), Object.is(0, -0), 0 === -0`

`true false false true false true false true` — `==` has a special rule for null; relational `>=` converts null to 0.

Q5. `[10, 1, 5, 100].sort()` and `[3, 20, 100].sort()` and `['b','a','C'].sort()`

`[1, 10, 100, 5]`, `[100, 20, 3]`, `['C', 'a', 'b']` — default sort compares UTF-16 strings; uppercase letters come before lowercase.

Q6. `['1', '2', '3'].map(parseInt)`

`[1, NaN, NaN]` — calls `parseInt('1',0)` (radix 0 → default 10), `parseInt('2',1)` (invalid radix), `parseInt('3',2)` ('3' isn't a binary digit).

Q7. `Math.max(), Math.min(), [] == ![], [] == false, !![]`

`-Infinity Infinity true true true` — max of nothing is the identity for max (−∞). `[]` is truthy but loosely equals false.

Q8. `parseInt('08'), parseInt('0x1F'), Number(''), Number(' 12 '), Number('12px'), parseFloat('12.5px'), +[], +{}, +[5], +[1,2]`

`8 31 0 12 NaN 12.5 0 NaN 5 NaN`

Q9. `'10' > '9', 10 > '9', 'B' < 'a', 1 < 2 < 3, 3 > 2 > 1`

`false true true true false` — two strings compare lexicographically; `3>2>1` → `true>1` → `1>1`.

Q10. `-7 % 3, 7 % -3, 5/0, 0/0, 1/-0`

`-1 1 Infinity NaN -Infinity` — remainder takes the sign of the dividend.

Q11. `'5' - '2', '5' + - '2', '5' - - '2', 'foo' * 1, [] * 2, [3] * [4]`

`3 '5-2' 7 NaN 0 12`

Q12. `('b' + 'a' + + 'a' + 'a').toLowerCase()`

`'banana'` — `+'a'` is NaN.

Q13. `9007199254740992 === 9007199254740993, Number.MAX_SAFE_INTEGER + 2, typeof 1n`

`true 9007199254740992 'bigint'` — beyond 2^{53} not every integer is representable.

Q14. `let i = 0; console.log(i++ + ++i, i);`

`2 2` — 0 (then i=1) + 2 (i=2).

Q15. `const a = {}, b = { k: 1 }, c = { k: 2 }; a[b] = 1; a[c] = 2; console.log(a[b]);`

`2` — both keys stringify to `'[object Object]'`. Use a Map.

Q16. `[..."👍a"].length, "👍a".length`

`2 3` — the emoji is a surrogate pair (2 code units); spread iterates code points.

Q17. `const re = /a/g; re.test('a'), re.test('a'), re.test('a')`

`true false true` — a global regex keeps `lastIndex` between calls.

Q18. `JSON.stringify({ a: undefined, b: () => 1, c: NaN, d: new Date(0), e: [undefined] })`

`{"c":null,"d":"1970-01-01T00:00:00.000Z","e":[null]}`

7.2 Hoisting, scope, closures

Q19.

```
console.log(typeof f, typeof g);
var f = function () {};
function g() {}
```

`undefined function` — `var f` is hoisted as undefined; the function declaration is hoisted with its body.

Q20.

```
var a = 1;
function b() { a = 10; return; function a() {} }
b();
console.log(a);
```

`1` — the hoisted `function a` creates a local `a` inside `b`, so `a = 10` assigns the local.

Q21.

```
var v = 'global';
function outer() { console.log(v); var v = 'local'; }
outer();
```

`undefined` — the local `var v` is hoisted and shadows the global.

Q22.

```
let x = 1;
{ console.log(x); let x = 2; }
```

`ReferenceError: Cannot access 'x' before initialization` — the inner `x` is in its TDZ; it does *not* fall back to the outer x.

Q23.

```
function f() { return typeof g; function g() {} var g = 1; }
console.log(f());
```

`'function'` — at the `return`, the assignment `g = 1` hasn't executed; the declaration of `var g` doesn't override the function.

Q24.

```
(function () { var q = r = 5; })();
console.log(typeof r, typeof q);
```

`number undefined` — parsed as `r = 5; var q = r;`; `r` becomes an implicit global (ReferenceError in strict mode).

Q25.

```
for (var i = 0; i < 3; i++) setTimeout(() => console.log(i), 0);
for (let j = 0; j < 3; j++) setTimeout(() => console.log(j), 0);
```

`3 3 3` then `0 1 2` — one shared `var` binding vs a fresh `let` binding per iteration. Pre-ES6 fix: an IIFE `(function(i){ … })(i)` or `setTimeout(console.log, 0, i)`.

Q26.

```
const fns = [];
for (let i = 0; i < 3; i++) { fns.push(() => i); i++; }
console.log(fns.map(f => f()));
```

`[1, 3]` — each iteration's binding is copied to the next iteration *before* the update expression; the closure sees its own iteration's binding after the inner `i++`.

Q27.

```
const counter = (() => { let c = 0; return () => c++; })();
counter(); counter();
console.log(counter());
```

`2` — postfix returns the old value.

Q28.

```
var fn = function g() { g = 1; return typeof g; };
console.log(fn());
```

`'function'` — the name of a named function expression is a read-only binding inside it (assignment silently ignored in sloppy mode, TypeError in strict).

7.3 References, arrays, functions

Q29.

```
function sq(arr) { arr.push(4); arr = [9]; return arr; }
const ar = [1];
console.log(sq(ar), ar);
```

`[9] [1, 4]` — mutation through the shared reference is visible; reassignment of the parameter is not.

Q30.

```
const ob = { a: 1, b: { c: 2 } };
const cp = { ...ob }; cp.b.c = 3; cp.a = 9;
console.log(ob);
```

`{ a: 1, b: { c: 3 } }` — spread is a shallow copy.

Q31. `const a = [1,2,3]; a[10] = 5; console.log(a.length, a.filter(() => true).length);`

`11 4` — `filter` skips holes.

Q32. `Array(3), Array(3).map(() => 1), Array.from({ length: 3 }, () => 1), [,,].length`

`[ <3 empty items> ], [ <3 empty items> ], [1, 1, 1], 2` — map skips holes; a trailing comma doesn't add an element.

Q33. `[1, 2, 3, 4].map(x => { if (x > 2) return x; })`

`[undefined, undefined, 3, 4]` — use `filter` to remove items.

Q34. `[].reduce((a, b) => a + b)`

`TypeError: Reduce of empty array with no initial value`

Q35. `function f(a, b = 2) { return arguments.length; } console.log(f(1), f(1, undefined), f.length);`

`1 2 1`

Q36. `[1, 2, 3].includes(NaN), [NaN].includes(NaN), [NaN].indexOf(NaN)`

`false true -1` — `includes` uses SameValueZero, `indexOf` uses `===`.

Q37. `function tf() { try { return 1; } finally { return 2; } } console.log(tf());`

`2` — `finally`'s return overrides.

Q38. `function Foo() { return { x: 1 }; } console.log(new Foo() instanceof Foo);`

`false` — a constructor that returns an object replaces the newly created instance.

7.4 `this`

Q39.

```
const o = { x: 1, getX() { return this.x; } };
console.log(o.getX(), (o.getX)(), (0, o.getX)());
```

`1 1 undefined` — parentheses alone keep the reference; the comma operator produces a plain function value, so `this` is lost (globalThis in sloppy mode; in a module/strict code it throws `TypeError: Cannot read properties of undefined`).

Q40.

```
'use strict';
const obj = {
  name: 'obj',
  regular() { return this.name; },
  arrow: () => typeof this,
  nested() { return [1].map(function () { return this; })[0]; },
  nestedArrow() { return [1].map(() => this.name)[0]; },
};
console.log(obj.arrow(), obj.nested(), obj.nestedArrow());
```

`'object'` `undefined` `'obj'` in a CommonJS file (top-level `this` is `module.exports`, an object); in an ES module `obj.arrow()` gives `'undefined'`; in a browser script, `window` → `'object'`. The callback in `nested` is a plain call → `undefined` in strict mode.

Q41.

```
function show() { return this.v; }
const b = show.bind({ v: 1 });
console.log(b.call({ v: 2 }), b.bind({ v: 3 })(), new (function () { this.v = 4; }.bind({ v: 5
}))().v);
```

`1` `1` `4` — `bind` is permanent against call/apply/bind, but `new` overrides it.

7.5 Event loop & promises

Q42.

```
console.log('1');
setTimeout(() => console.log('2'), 0);
Promise.resolve().then(() => console.log('3')).then(() => console.log('4'));
queueMicrotask(() => console.log('5'));
(async () => { console.log('6'); await null; console.log('7'); })();
process.nextTick(() => console.log('8'));
console.log('9');
```

`1` `6` `9` `8` `3` `5` `7` `4` `2` — sync first (async function body runs synchronously until the first await), then nextTick, then microtasks FIFO (3,5,7 were queued in that order; 4 is queued when 3 runs), then the timer.

Q43.

```
async function a1() { console.log('a1 start'); await a2(); console.log('a1 end'); }
async function a2() { console.log('a2'); }
console.log('script start');
setTimeout(() => console.log('timeout'), 0);
a1();
new Promise(r => { console.log('p1'); r(); }).then(() => console.log('p2'));
console.log('script end');
```

`script start → a1 start → a2 → p1 → script end → a1 end → p2 → timeout` — the Promise executor runs synchronously.

Q44.

```
Promise.resolve().then(() => {
  console.log('m1');
  setTimeout(() => console.log('to2'), 0);
  Promise.resolve().then(() => console.log('m2'));
});
setTimeout(() => console.log('to'), 0);
```

`m1 m2 to to2` — microtasks queued by a microtask still run before any timer; `to2` was scheduled after `to`.

Q45. `new Promise((res, rej) => { res(1); rej(2); res(3); }).then(console.log);`

`1` — a promise settles only once; later calls are ignored.

Q46. `Promise.resolve(1).then(2).then(console.log); Promise.resolve(1).finally(() => 2).then(console.log);`

`1` and `1` — a non-function passed to `then` is ignored (value passes through); `finally`'s return value is ignored.

Q47. `async function af() { return 1; } console.log(af());`

`Promise { 1 }` — async functions always return promises.

Q48.

```
const sleep = ms => new Promise(r => setTimeout(r, ms));
async function seq() { await sleep(100); await sleep(100); }
async function par() { await Promise.all([sleep(100), sleep(100)]); }
```

How long does each take?

~200 ms vs ~100 ms. Awaiting in sequence serializes independent work.

Q49.

```
try {
  setTimeout(() => { throw new Error('boom'); }, 0);
} catch (e) { console.log('caught'); }
```

Never prints “caught”; the error is thrown later from a different task (uncaught exception, crashes Node). Same for a promise rejection inside `try` that isn't awaited.

Part 8 — Hard Coding Challenges (with tested solutions)

These are the implementation questions senior JS interviews actually ask. Attempt each with a 20–30 minute timer. All solutions below pass tests in Node 22.

Challenge list

1. closures `debounce(fn, ms)` and `throttle(fn, ms)` preserving `this` and arguments.
2. functional `curry(fn)` supporting `f(1)(2)(3)`, `f(1,2)(3)`, `f(1)(2,3)`.
3. functional `pipe` and `compose`.
4. recursion `flatten(arr, depth)` without `Array.prototype.flat`.
5. hard `deepClone` that handles cycles, Dates, RegExps, Maps, Sets, symbol keys and prototypes.
6. hard `deepEqual(a, b)` treating NaN as equal and checking prototypes.
7. promises Implement `Promise.all` and `Promise.allSettled`.
8. this Polyfill `Function.prototype.bind` including the `new` case.
9. arrays Polyfill `Array.prototype.reduce` (holes, no initial value, empty array).
10. closures `memoize(fn)` and use it to compute `fib(90)` instantly.
11. classes `EventEmitter` with `on`, `off`, `once`, `emit`.
12. data structures `LRUCache` with O(1) `get` / `put`.
13. async `mapLimit(items, limit, worker)`: at most `limit` tasks in flight, results in order.
14. async `retry(fn, {retries, delay, factor})` with exponential backoff; `withTimeout(promise, ms)`.
15. objects `get(obj, 'a.b[0].c', fallback)` (lodash-style path access).
16. recursion `permutations(arr)`; `isBalanced('{[()]}')`.
17. generators `range(start, end, step)` as a lazy generator.
18. metaprogramming `reactive(obj, onChange)` using Proxy.

Solutions

1. debounce & throttle

```
function debounce(fn, wait) {
  let timer;
  return function (...args) {
    clearTimeout(timer);
    timer = setTimeout(() => fn.apply(this, args), wait);   // arrow keeps the wrapper's this
  };
}
function throttle(fn, wait) {
  let last = 0;
  return function (...args) {
    const now = Date.now();
    if (now - last >= wait) { last = now; return fn.apply(this, args); }
  };
}
// Follow-ups: leading/trailing options, a cancel() method, throttle that runs the last skipped
call.
```

2–3. curry, pipe, compose

```
function curry(fn) {
  return function curried(...args) {
    return args.length >= fn.length
      ? fn.apply(this, args)
      : (...more) => curried.apply(this, [...args, ...more]);
  };
}
const pipe = (...fns) => x => fns.reduce((v, f) => f(v), x);         // left → right
const compose = (...fns) => x => fns.reduceRight((v, f) => f(v), x); // right → left
// pipe(x => x + 1, x => x * 2)(3) === 8 ; compose(...)(3) === 7
```

4. flatten

```
function flatten(arr, depth = Infinity) {
  return depth < 1 ? arr.slice() : arr.reduce((acc, x) =>
    Array.isArray(x) ? acc.concat(flatten(x, depth - 1)) : acc.concat([x]), []);
}
// Iterative alternative for huge depth: use an explicit stack and reverse at the end.
```

5. deepClone

```
function deepClone(value, seen = new WeakMap()) {
  if (value === null || typeof value !== 'object') return value;   // primitives & functions
  if (seen.has(value)) return seen.get(value);                       // cycles / shared refs
  if (value instanceof Date) return new Date(value);
  if (value instanceof RegExp) return new RegExp(value.source, value.flags);
  if (value instanceof Map) {
    const m = new Map(); seen.set(value, m);
    value.forEach((v, k) => m.set(deepClone(k, seen), deepClone(v, seen)));
    return m;
  }
  if (value instanceof Set) {
    const s = new Set(); seen.set(value, s);
    value.forEach(v => s.add(deepClone(v, seen)));
    return s;
  }
  const copy = Array.isArray(value) ? [] : Object.create(Object.getPrototypeOf(value));
  seen.set(value, copy);                        // register BEFORE recursing
  for (const key of Reflect.ownKeys(value)) copy[key] = deepClone(value[key], seen);  // incl.
symbols
  return copy;
}
```

6. deepEqual

```
function deepEqual(a, b) {
  if (Object.is(a, b)) return true;
  if (typeof a !== 'object' || typeof b !== 'object' || a === null || b === null) return false;
  if (Object.getPrototypeOf(a) !== Object.getPrototypeOf(b)) return false;   // [1] vs {0:1}
  const ka = Reflect.ownKeys(a), kb = Reflect.ownKeys(b);
  if (ka.length !== kb.length) return false;
  return ka.every(k => Object.prototype.hasOwnProperty.call(b, k) && deepEqual(a[k], b[k]));
}
// Follow-ups: cycles (track pairs in a WeakMap), Dates/Maps/Sets, typed arrays.
```

7. Promise.all / allSettled

```
function promiseAll(iterable) {
  return new Promise((resolve, reject) => {
    const items = Array.from(iterable);
    const results = new Array(items.length);
    let remaining = items.length;
    if (remaining === 0) return resolve(results);          // edge case: empty input
    items.forEach((item, i) => {
      Promise.resolve(item).then(value => {                 // accept non-promises / thenables
        results[i] = value;                                 // keep INPUT order, not completion
order
        if (--remaining === 0) resolve(results);
      }, reject);                                           // first rejection wins
    });
  });
}
function promiseAllSettled(iterable) {
  return promiseAll(Array.from(iterable, p => Promise.resolve(p).then(
    value => ({ status: 'fulfilled', value }),
    reason => ({ status: 'rejected', reason }))));
}
```

8. bind polyfill

```
Function.prototype.myBind = function (ctx, ...bound) {
  const fn = this;
  function boundFn(...args) {
    // when called with new, `this` is an instance of boundFn → ignore ctx
    return fn.apply(this instanceof boundFn ? this : ctx, [...bound, ...args]);
  }
  boundFn.prototype = Object.create(fn.prototype);   // so instances are instanceof fn
  return boundFn;
};
```

9. reduce polyfill

```
Array.prototype.myReduce = function (cb, ...init) {
  let i = 0, acc;
  const len = this.length;
  if (init.length) acc = init[0];               // distinguishes "no initial" from "initial =
undefined"
  else {
    while (i < len && !(i in this)) i++;         // skip holes
    if (i >= len) throw new TypeError('Reduce of empty array with no initial value');
    acc = this[i++];
  }
  for (; i < len; i++) if (i in this) acc = cb(acc, this[i], i, this);
  return acc;
};
```

10. memoize

```
function memoize(fn, keyFn = (...args) => JSON.stringify(args)) {
  const cache = new Map();
  return function (...args) {
    const key = keyFn(...args);
    if (!cache.has(key)) cache.set(key, fn.apply(this, args));
    return cache.get(key);
  };
}
const fib = memoize(n => (n < 2 ? n : fib(n - 1) + fib(n - 2)));
fib(90);   // 2880067194370816000 — instantly (beyond 2^53, so not exact; use BigInt for
exactness)
```

11. EventEmitter

```
class EventEmitter {
  #events = new Map();
  on(name, fn) {
    if (!this.#events.has(name)) this.#events.set(name, []);
    this.#events.get(name).push(fn);
    return () => this.off(name, fn);              // unsubscribe function
  }
  off(name, fn) {
    const list = this.#events.get(name);
    if (list) this.#events.set(name, list.filter(f => f !== fn && f.original !== fn));
  }
  once(name, fn) {
    const wrapper = (...args) => { this.off(name, wrapper); fn(...args); };
    wrapper.original = fn;                         // so off(name, fn) also removes a once-
listener
    return this.on(name, wrapper);
  }
  emit(name, ...args) {
    const list = this.#events.get(name);
    if (!list) return false;
    [...list].forEach(fn => fn(...args));          // copy: listeners may unsubscribe during emit
    return true;
  }
}
```

12. LRU cache (Map preserves insertion order)

```
class LRUCache {
  constructor(capacity) { this.capacity = capacity; this.map = new Map(); }
  get(key) {
    if (!this.map.has(key)) return -1;
    const value = this.map.get(key);
    this.map.delete(key); this.map.set(key, value);   // move to "most recent"
    return value;
  }
  put(key, value) {
    this.map.delete(key);
    this.map.set(key, value);
    if (this.map.size > this.capacity) this.map.delete(this.map.keys().next().value);  // evict
oldest
  }
}
// Interview follow-up: implement it with a hash map + doubly linked list (language-agnostic
version).
```

13. Concurrency limiter

```
async function mapLimit(items, limit, worker) {
  const results = new Array(items.length);
  let next = 0;                                    // shared cursor; safe because JS is single-
threaded
  async function run() {
    while (next < items.length) {
      const i = next++;
      results[i] = await worker(items[i], i);
    }
  }
  await Promise.all(Array.from({ length: Math.min(limit, items.length) }, run));
  return results;
}
```

14. retry with backoff, timeout

```
async function retry(fn, { retries = 3, delay = 100, factor = 2 } = {}) {
  for (let attempt = 0; ; attempt++) {
    try { return await fn(attempt); }
    catch (err) {
      if (attempt >= retries) throw err;
      await new Promise(r => setTimeout(r, delay * factor ** attempt));   // add random jitter in
production
    }
  }
}
function withTimeout(promise, ms) {
  let id;
  const timeout = new Promise((_, reject) => { id = setTimeout(() => reject(new
Error('Timeout')), ms); });
  return Promise.race([promise, timeout]).finally(() => clearTimeout(id));   // don't leak the
timer
}
```

15. get by path

```
function get(obj, path, fallback) {
  const keys = Array.isArray(path) ? path
    : path.replace(/\[(\w+)\]/g, '.$1').split('.').filter(Boolean);
  let cur = obj;
  for (const k of keys) { if (cur == null) return fallback; cur = cur[k]; }
  return cur === undefined ? fallback : cur;
}
```

16–17. permutations, balanced brackets, range

```
function permutations(arr) {
  if (arr.length <= 1) return [arr.slice()];
  return arr.flatMap((x, i) =>
    permutations([...arr.slice(0, i), ...arr.slice(i + 1)]).map(p => [x, ...p]));
}
function isBalanced(s) {
  const pairs = { ')': '(', ']': '[', '}': '{' }, stack = [];
  for (const ch of s) {
    if ('([{'.includes(ch)) stack.push(ch);
    else if (ch in pairs && stack.pop() !== pairs[ch]) return false;
  }
  return stack.length === 0;
}
function* range(start, end, step = 1) { for (let i = start; i < end; i += step) yield i; }
```

18. reactive with Proxy

```
function reactive(target, onChange) {
  return new Proxy(target, {
    set(obj, key, value, receiver) {
      const old = obj[key];
      const ok = Reflect.set(obj, key, value, receiver);
      if (old !== value) onChange(key, old, value);
      return ok;            // must return true, or a TypeError is thrown in strict mode
    },
  });
}
```

Extra challenges (no solution given — your homework)

1. Implement `Promise` itself (`MyPromise` with `then` chaining, thenable adoption, async resolution via `queueMicrotask`). Pass the Promises/A+ test suite.
2. Implement a tiny `render(vnode, container)` virtual-DOM with a diff/patch step.
3. Write a JSON parser (recursive descent) without `JSON.parse`.
4. Implement an async task queue with priorities, pause/resume and cancellation via `AbortSignal`.
5. Implement `Array.prototype.sort` as a stable merge sort that accepts a comparator.
6. Create a `classNames()` utility, a `deepMerge()`, and an `Observable` with `map` / `filter` / `subscribe`.
7. Build a `Router` for a single-page app using the History API.
8. Write a function that detects whether a linked list (objects with `next`) has a cycle, in O(1) memory.
9. Implement `sleepSort` and explain exactly why it is not reliable.

10. Implement `Function.prototype.call` without using `call`, `apply` or `bind` (hint: temporary Symbol key on the context object).

Part 9 — Conceptual Interview Questions (with model answers)

Language core

1. Explain `var` vs `let` vs `const`, hoisting, and the TDZ.

All three are hoisted. `var` is function-scoped and initialized to `undefined`; `let`/`const` are block-scoped and stay uninitialized until their declaration runs — the TDZ, where access throws ReferenceError. `const` prevents rebinding, not mutation. `let` in loops creates a binding per iteration.

2. What is a closure? Give a real use and a pitfall.

A function plus a reference to the lexical environment where it was created; it can read/write those variables after the outer function returns. Uses: private state, factories, memoization, debounce, partial application. Pitfalls: `var` in loops (shared binding), memory retention of large captured objects, stale values in React hooks.

3. How is `this` determined?

By the call site, in priority: `new` → explicit (`call/apply/bind`) → implicit (`obj.method()`) → default (`undefined` in strict, global in sloppy). Arrow functions have no own `this`; they capture it lexically. Class bodies are strict, so detached methods get `undefined`.

4. Explain prototypal inheritance and how `class` relates to it.

Objects delegate failed property lookups to their `[[Prototype]]`, forming a chain ending in `null`. `class` is mostly syntax: methods go on `C.prototype`, `extends` sets both `C.prototype.__proto__ = Parent.prototype` and `C.__proto__ = Parent` (static inheritance). Differences from function constructors: strict mode, TDZ, non-enumerable methods, must use `new`, `super`, private `#fields`.

5. `==` vs `===` vs `Object.is`.

`===` compares without coercion (NaN≠NaN, +0===−0). `==` coerces with the abstract equality algorithm (null==undefined only with each other; booleans → numbers; objects → primitives). `Object.is` is SameValue: NaN is NaN, +0 is not −0. Use `===`; `x == null` is the only common idiomatic `==`.

6. Pass by value or by reference?

Always by value; for objects the value is a reference (“call by sharing”). You can mutate the object through the parameter, but reassigning the parameter doesn't affect the caller.

7. Shallow vs deep copy — list ways to do each and their limitations.

Shallow: spread, `Object.assign`, `slice`, `Array.from`. Deep: `structuredClone` (handles cycles, Date, Map, Set, typed arrays; fails on functions/DOM nodes, loses prototypes/class identity and getters), JSON round-trip (loses undefined, functions, symbols, Dates become strings, NaN → null, fails on cycles/BigInt), custom recursive clone.

8. Why is `typeof null === 'object'`?

A bug from the first implementation: values were tagged with type bits, object tag was 0, and null was the NULL pointer (all zeros). Fixing it would break the web, so it remains.

9. What happens when you call `new F()`?

Create an empty object whose prototype is `F.prototype`; call `F` with `this` bound to it; if `F` returns an object, that's the result, otherwise the new object is.

10. Function declarations vs expressions vs arrows — when to use which?

Declarations: hoisted, good for top-level named utilities. Expressions: conditional definitions, closures, not hoisted. Arrows: concise callbacks, lexical `this` / `arguments`, not constructible; don't use them as object methods that need `this` or as prototype methods.

Async

11. Describe the event loop, microtasks vs macrotasks.

One call stack. After each macrotask (script, timer, I/O, event), the engine drains the entire microtask queue (promise reactions, await continuations, queueMicrotask), then the browser may render, then takes the next macrotask. Therefore promise callbacks always run before `setTimeout(…, 0)`, and an endless chain of microtasks starves rendering and timers.

12. Is JavaScript single-threaded? Then how is `fetch` non-blocking?

The JS execution of an agent is single-threaded, but the host (browser/libuv in Node) performs I/O on other threads or via OS async APIs and enqueues callbacks when done. Parallel JS needs Workers.

13. What does `async/await` compile to conceptually? Error handling?

A state machine over promises (like a generator driven by a runner). `await` suspends the function and resumes it in a microtask when the promise settles; a rejection is thrown at the `await` point, so you use try/catch. An async function returns a promise that rejects if the function throws.

14. Difference between `Promise.all`, `allSettled`, `race`, `any`? Does `Promise.all` cancel the others on failure?

See table in 6.5. No — promises are not cancellable; the others keep running; their results are ignored. Use `AbortController` to actually cancel the underlying work.

15. How would you handle a race condition when a user types fast in a search box?

Debounce the input; abort the previous request with `AbortController`; or tag requests with an incrementing id and ignore responses that are not the latest.

Engine, memory, performance

16. How does garbage collection work in JS and how do leaks happen?

Reachability: objects reachable from roots (globals, stack, active closures) are kept; mark-and-sweep collects the rest (V8: generational — young gen scavenger, old gen mark-compact, incremental/concurrent). Leaks are unintentional reachability: globals, uncleared timers/listeners, growing caches, detached DOM nodes held in JS, closures capturing big data. Cycles alone are *not* leaks (unlike ref-counting).

17. What are hidden classes / inline caches, and why should you care?

V8 assigns objects "shapes" based on property order; inline caches at property access sites specialize to seen shapes. Monomorphic sites are fast; megamorphic ones (many shapes) are slow. Initialize all fields in the constructor in the same order; avoid `delete` on hot objects; don't mix element types in arrays.

18. Why is `0.1 + 0.2 !== 0.3` and how do you handle money?

Binary floating point can't represent those decimals exactly. Store integer minor units (cents), use BigInt, or a decimal library; round only for display with `Intl.NumberFormat`.

Modules, tooling, browser

19. ESM vs CommonJS?

ESM is static (imports known before execution → tree shaking), async loading, live bindings, strict, top-level await. CJS is dynamic synchronous `require`, exports a value copy, used by older Node code. Interop: ESM can import CJS; CJS can `require()` synchronous ESM in recent Node versions, or use dynamic `import()`.

20. Event bubbling, capturing and delegation.

Events travel capture (root → target), target, bubble (target → root). Delegation attaches one listener on an ancestor and uses `event.target.closest(selector)` to handle many children, including ones added later — fewer listeners, less memory.

21. How do you prevent XSS in JS code?

Never put untrusted data into `innerHTML`, `outerHTML`, `document.write`, `eval`, `setTimeout(string)`, or `href="javascript:…"`. Use `textContent`, framework escaping, sanitize with DOMPurify when HTML is required, and set a Content-Security-Policy.

22. `defer` vs `async` on scripts?

Both download in parallel without blocking parsing. `defer` executes after parsing, in document order, before `DOMContentLoaded`. `async` executes as soon as it downloads, in any order. Modules are deferred by default.

23. What is the difference between `null` and `undefined`, and when would each appear?

`undefined`: language default for missing values (uninitialized variable, missing property/argument, no return). `null`: explicit "no value" assigned by code/APIs (e.g., `querySelector` returns null). `typeof` differs; `==` treats them as equal; default parameters only kick in for undefined.

24. Explain `Symbol.iterator` and how `for…of`, spread and destructuring use it.

Any object with a `[Symbol.iterator]()` method returning an iterator (`next() → {value, done}`) is iterable. `for…of`, spread `[...x]`, array destructuring, `Array.from`, `Promise.all`, `new Map(x)` all consume that protocol. Generators are the easiest way to implement it.

25. Design question: how would you structure a medium-size vanilla JS app?

ES modules by feature; pure logic separated from DOM/IO (testable); a single state store with explicit update functions (or an event emitter); event delegation; fetch wrapper with error handling, timeouts and abort; JSDoc/TypeScript types; ESLint + Prettier; tests with `node:test`/Vitest; build with Vite.

Part 10 — Cheat Sheet

Topic	Remember
Declarations	`const` > `let` > never `var`; TDZ; function declarations hoist fully
Types	7 primitives + object; `typeof null` = 'object'; `Array.isArray`; `Number.isNaN`
Falsy	`false 0 -0 0n "" null undefined NaN` — everything else truthy (`[]`, `{}`, `"0"`)
Equality	use `===`; `x == null` for null/undefined; NaN ≠ NaN; `Object.is`
Defaults	`??` for null/undefined, `||` for any falsy; `?.` for safe access
Numbers	doubles; safe int $2^{53}-1$; `toFixed` returns string; `%` keeps dividend sign; BigInt for big ints
Strings	immutable; UTF-16 length; template literals; `slice`, `at(-1)`, `localeCompare`
Arrays	`sort` is lexicographic & mutating → `(a,b)=>a-b` or `toSorted`; `reduce` with initial value; spread is shallow
Loops	`for…of` values, `for…in` keys (objects only); can't break `forEach`; `forEach` doesn't await
Functions	arrows: lexical `this`, no `arguments`, no `new`; defaults only for undefined; rest/spread
`this`	new > bind/call/apply > obj.method() > plain call (undefined in strict)
Event loop	sync → nextTick (Node) → all microtasks → render → one macrotask → repeat
Async	`Promise.all` for parallel; try/catch around await; always handle rejections; AbortController to cancel
Copying	`structuredClone` for deep; `{...o}` / `[...a]` shallow
Collections	Map for dynamic keys, Set for uniqueness, WeakMap for object metadata
Security	`textContent` not `innerHTML`; no `eval`; `crypto.getRandomValues` for secrets

What to learn next (in order)

1. **TypeScript** — generics, union/intersection types, narrowing, utility types.
2. **DOM + browser APIs** — build 3 small vanilla apps (todo with localStorage, weather app with fetch, a game with canvas).
3. **Node.js** — fs, streams, http, Express/Fastify, REST, environment config, testing.
4. **A framework** — React (with hooks) is the most common in job listings.
5. **Reading:** *You Don't Know JS Yet* (Kyle Simpson), *javascript.info*, MDN Web Docs, *Eloquent JavaScript*, the ECMAScript spec for deep questions.

Prepared for Mohamad — JavaScript Foundations fast track. Verified against Node.js v22. Browser behavior may differ only where noted (e.g. `process.nextTick`, `setImmediate`, top-level `this`).